MODULE 25 · DOMAIN 6 - SECURITY AND COMPLIANCE

# Secrets, Keys and Certificates

Secrets Manager and Parameter Store, rotation and sharing, KMS key policies, grants and multi-Region keys, CloudHSM, and ACM certificates.

DOMAIN 6 of 6 EXAM WEIGHT 17% MODULE 25 of 34

admin@sthithapragna.com

A secret should be **fetched at run time**, **rotated** without anyone ever seeing it, and protected by a **key whose policy you control**.

# Secrets Manager or Parameter Store?

SECRETS MANAGER

## Built for secrets

- Automatic rotation, managed or with Lambda
- Replication to other Regions
- Resource policies for cross-account access
- Charged per secret per month

PARAMETER STORE

## Built for configuration

- SecureString values encrypted with KMS
- No built-in rotation
- Standard tier free; advanced tier adds policies and RAM sharing
- Can reference Secrets Manager secrets

THE TIEBREAKER

'Must rotate automatically' points to Secrets Manager. Plain configuration, or free and simple: Parameter Store.

# How rotation works

01

## Create

A new secret value is generated and staged as AWSPENDING.

02

## Set

The database or service is updated with the new value.

03

## Test

The new value is tried against the service.

04

## Finish

AWSPENDING becomes AWSCURRENT; the old becomes AWSPREVIOUS.

ON THE EXAM

Managed rotation handles these steps for RDS, Aurora, Redshift and DocumentDB with no Lambda to maintain.

# Rotation in practice

- Schedules use a rate or a cron expression, with a rotation window.
- Applications should fetch the secret, cache it briefly, and fetch again on authentication failure.
- A custom rotation Lambda in a VPC needs a path to both the database and Secrets Manager.
- Hard-coding a secret's value anywhere breaks the moment it rotates.

> ON THE EXAM
>
> Rotation fails with a timeout: the rotation function cannot reach Secrets Manager or the database.

# Sharing secrets and parameters

- **Cross-account secret**: a resource policy on the secret, plus the caller's identity policy.
- The secret must use a **customer managed key** whose policy admits the other account.
- **Replicate** secrets to other Regions for disaster recovery and local reads.
- **Advanced-tier parameters** can be shared with other accounts through AWS RAM.

> ON THE EXAM
>
> Cross-account secret denied despite a correct resource policy: the secret uses the AWS managed key.

# Three kinds of KMS key

| KIND | WHO CONTROLS IT | CROSS-ACCOUNT? |
|---|---|---|
| AWS owned | AWS, invisibly, across many accounts | Not applicable |
| AWS managed (aws/s3, aws/rds...) | AWS; you see it but cannot change its policy | No |
| Customer managed | You: policy, rotation, deletion | Yes, through the key policy |

ON THE EXAM

Anything cross-account, custom rotation, or auditable key control needs a customer managed key.

# A key policy, in three parts

```
1 { "Sid": "EnableIAMPolicies", "Effect": "Allow",
2   "Principal": { "AWS": "arn:aws:iam::111122223333:root" },
3   "Action": "kms:*", "Resource": "*" },
4 { "Sid": "KeyAdministrators", "Effect": "Allow",
5   "Principal": { "AWS": "arn:aws:iam::111122223333:role/KeyAdmins" },
6   "Action": ["kms:Create*", "kms:Put*", "kms:ScheduleKeyDeletion"], "Resource": "*" },
7 { "Sid": "KeyUsers", "Effect": "Allow",
8   "Principal": { "AWS": "arn:aws:iam::444455556666:role/Reader" },
9   "Action": ["kms:Decrypt", "kms:DescribeKey"], "Resource": "*" }
```

2 Lets IAM policies in this account grant use of the key.

8 A role in another account may decrypt.

ON THE EXAM

Without the first statement, IAM policies alone can never grant use of the key: the key policy is primary.

# Grants

- A **grant** gives a principal permission to use a key, without editing the key policy.
- Grants are created and retired or revoked programmatically, for temporary delegation.
- AWS services such as EBS and RDS use grants to act on your keys for you.
- The condition kms:GrantIsForAWSResource limits grant creation to AWS services.

> ON THE EXAM
>
> Temporary, programmatic, revocable use of a key for a specific principal: a grant, not repeated policy edits.

# Rotation and multi-Region keys

- Automatic rotation creates new key material; old material is kept, so old data still decrypts.
- The rotation period is configurable, and rotation can also be done on demand.
- **Multi-Region keys** share key material and ID across Regions: encrypt in one, decrypt in another.
- Keys with imported key material do not rotate automatically. Deleting a key waits 7 to 30 days.

> ON THE EXAM
>
> Data encrypted in one Region must be decrypted in another without re-encrypting: multi-Region keys.

# Narrowing key use with conditions

- **kms:ViaService**: the key may only be used through a named service, such as S3 or Secrets Manager.
- **Encryption context**: extra key-value data that must match to decrypt, and appears in CloudTrail.
- **kms:CallerAccount**: limit use to principals from specific accounts.
- Disable a key rather than delete it if you might ever need the data again.

> ON THE EXAM
>
> Allow decryption only when data is read through S3, never directly: kms:ViaService.

# CloudHSM and custom key stores

- **CloudHSM**: single-tenant hardware security modules in your VPC; you manage the HSM users.
- For compliance that demands dedicated HSMs, or keys under your exclusive control.
- A **KMS custom key store** backed by CloudHSM keeps KMS integrations with your own HSMs.
- An **external key store** keeps key material outside AWS entirely.

> ON THE EXAM
>
> Dedicated, single-tenant HSMs you control, but still using KMS integrations: a CloudHSM-backed custom key store.

# AWS Certificate Manager

- Free public certificates for integrated services: load balancers, CloudFront, API Gateway.
- Renewed automatically; DNS validation lets renewal happen with no human involved.
- Certificates are Regional. **CloudFront needs its certificate in us-east-1**.
- Since February 2026, new certificates last at most **198 days**, and renew 45 days before expiry.

> ON THE EXAM
>
> Imported certificates are not renewed by ACM. Watch their expiry with EventBridge events or a Config rule.

# Certificates outside AWS, and private certificates

- **Exportable public certificates** (June 2025) can be exported with their private key, for use anywhere.
- ACM still renews them; an EventBridge event on renewal can trigger redeployment.
- **AWS Private CA** issues private certificates for internal services and devices.
- Certificates issued before 17 June 2025 cannot be exported.

> ON THE EXAM
>
> Public certificate for on-premises servers, renewed automatically by ACM: exportable certificates plus renewal automation.

> An application connects to an Amazon RDS for MySQL database with a password stored in a configuration file. Auditors require the password to rotate every 30 days automatically.
>
> The team wants the least operational overhead, and no rotation code to maintain.

## What should the DevOps engineer do?

A Store the password as a Parameter Store SecureString, and write a Lambda function to rotate it

B Have a DBA change the password monthly and update the file

C Store the password in Secrets Manager with managed rotation on a 30-day schedule, and have the app fetch it at run time

D Encrypt the configuration file with a customer managed KMS key

◆ PAUSE AND DECIDE

## What should the DevOps engineer do?

A Parameter Store and a Lambda

*Works, but it is exactly the rotation code the team does not want to maintain.*

B Monthly by hand

*Not automatic, and a person sees the password.*

C Secrets Manager managed rotation

Rotates on schedule with no code; the app always fetches the current value.

D Encrypt the file

*Protects the file. The password still never rotates.*

THE SIGNAL

Automatic rotation, no code, RDS: Secrets Manager managed rotation.

> A Lambda function in account B must read a secret in account A. The secret's resource policy allows the function's role, and the role's identity policy allows secretsmanager:GetSecretValue.
>
> Calls still fail with access denied. The secret is encrypted with the default aws/secretsmanager key.

## What should the DevOps engineer do?

A. Add another statement to the function role's identity policy

B. Share the secret through AWS RAM

C. Make the secret's resource policy allow all principals

D. Re-encrypt the secret with a customer managed key whose policy allows the role to decrypt, and grant kms:Decrypt to the role

◆ PAUSE AND DECIDE

## What should the DevOps engineer do?

A More identity policy

*The identity side already allows it. The key is refusing.*

B RAM sharing

*Secrets are shared by resource policy, not RAM, and the key still refuses.*

C Allow everyone

*Dangerous, and still blocked by the key.*

D Customer managed key

The AWS managed key cannot admit another account. A customer managed key can.

THE SIGNAL

Two locks confirmed open, a default key named: the third lock, the key, is the problem.

> A batch system launches jobs that each need to decrypt data with a customer managed KMS key, using a job-specific role. Permission must exist only while the job runs.
>
> The security team does not want the key policy edited for every job.

## What should the DevOps engineer use?

A Edit the key policy to add and remove each job's role

B Add kms:Decrypt to every job role's identity policy permanently

C Create a KMS grant for the job's role when the job starts, and retire it when the job finishes

D Create a new KMS key for every job

◆ PAUSE AND DECIDE

## What should the DevOps engineer use?

A Edit the key policy

*Ruled out, and risky to edit constantly.*

B Permanent permission

*Fails 'only while the job runs'.*

C Grant, then retire

Programmatic, temporary delegation with no key policy edits: what grants are for.

D A key per job

*Data is encrypted under one key. New keys cannot decrypt it.*

THE SIGNAL

Temporary, programmatic, revocable key use: grants.

> Application data is encrypted client-side with a KMS key in us-east-1 and replicated to us-west-2 for disaster recovery.
>
> During a failover test, the application in us-west-2 cannot decrypt the replicated data, because the key exists only in us-east-1. The company wants to decrypt locally in either Region, without re-encrypting data.

**What should the DevOps engineer use?**

A Copy the key to us-west-2

B Create a new key in us-west-2 with the same alias

C A grant that lets us-west-2 principals use the us-east-1 key

D A multi-Region primary key in us-east-1 with a replica key in us-west-2

◆ PAUSE AND DECIDE

## What should the DevOps engineer use?

A **Copy the key**
*Standard KMS keys cannot be copied or exported.*

B **Same alias**
*A different key underneath. It cannot decrypt the data.*

C **Cross-Region grant**
*Calls would still go to us-east-1, which fails in a Regional outage.*

D **Multi-Region keys**
Same key material and ID in both Regions: decrypt locally, no re-encryption.

THE SIGNAL

Decrypt the same ciphertext in another Region, locally: multi-Region keys.

> A team requests an ACM certificate for www.example.com in eu-west-2, where its application runs, and validates it by DNS.
>
> When configuring a CloudFront distribution in front of the application, the certificate does not appear in the list of choices.

## What should the team do?

A. Request a certificate for the domain in us-east-1, and select it in CloudFront

B. Export the eu-west-2 certificate and upload it to CloudFront

C. Switch the certificate from DNS validation to email validation

D. Wait for the certificate to finish propagating to all Regions

◆ PAUSE AND DECIDE

## What should the team do?

**A** Request it in us-east-1

CloudFront only uses ACM certificates from us-east-1.

**B** Export and upload

*Unnecessary effort, and loses ACM's automatic renewal for CloudFront.*

**C** Email validation

*Validation method is not the issue, and DNS is the better choice anyway.*

**D** Wait to propagate

*ACM certificates are Regional. They never propagate.*

THE SIGNAL

CloudFront certificate missing: it is in the wrong Region. CloudFront wants us-east-1.

> A company runs several public-facing web servers on premises. Their TLS certificates, bought from a third party, are renewed by hand and expired twice last year.
>
> The company wants publicly trusted certificates for those servers that renew automatically, with the fewest moving parts.

## What should the DevOps engineer do?

A Issue certificates from AWS Private CA

B Use ACM exportable public certificates, with an EventBridge rule on renewal that redeploys them through Systems Manager

C Generate self-signed certificates with a long validity

D Import the third-party certificates into ACM so that ACM renews them

◆ PAUSE AND DECIDE

## What should the DevOps engineer do?

A Private CA

*Private certificates are not trusted by the public's browsers.*

B Exportable certificates with renewal automation

Publicly trusted, renewed by ACM, redeployed automatically on each renewal.

C Self-signed

*Browsers do not trust them.*

D Import into ACM

*ACM never renews imported certificates.*

THE SIGNAL

Public trust, on premises, automatic renewal: exportable certificates, plus automation on the renewal event.

# Module 25 in five lines

- Fetch at run time, rotate without anyone seeing, protect with a key you control.
- Rotation needed: Secrets Manager, managed for RDS. Plain config: Parameter Store.
- Cross-account secrets and data need customer managed keys; AWS managed keys cannot be shared.
- Grants for temporary key use; multi-Region keys for decrypting in another Region.
- ACM: CloudFront needs us-east-1; 198-day certificates; imported ones never auto-renew; exportable since June 2025.

> ON THE EXAM
>
> Next, module 26: network defence in depth.